

Урок 28. Корутины. Flow. SharedFlow и StateFlow

11 декабря 2020

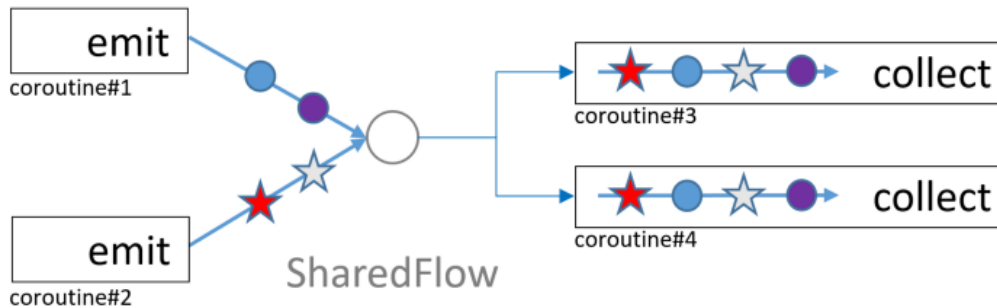
В этом уроке подробно разбираем что такое SharedFlow и StateFlow.

SharedFlow

Давайте вспомним, как работает обычный Flow. Он начинает работу, когда получатель вызывает его метод collect. И для каждого нового получателя, который вызывает метод collect, создается и начинает работать новый Flow. Такое поведение называется cold.

SharedFlow отличается от обычного Flow тем, что он не cold, а hot. Он работает в одном экземпляре, его старт не привязан к получателям, и он может иметь несколько получателей. Самый популярный пример использования - Event Bus. Ранее для этих целей использовался BroadcastChannel, а теперь ему на замену пришел SharedFlow.

Схематично это выглядит так:



[Источник](#)

С одной стороны может быть несколько отправителей данных, а с другой - несколько получателей. Каждый получатель будет получать все данные от всех отправителей.

Все это потокобезопасно и не требует от нас отдельных действий по синхронизации.

Давайте посмотрим, как работать с SharedFlow. Для этого нам понадобится core версии 1.4.2:

```
1 | implementation "org.jetbrains.kotlinx:kotlinx-coroutines-core:1.4.2"
```

У SharedFlow есть два отдельных интерфейса: MutableSharedFlow - для отправителей, SharedFlow - для получателей (аналогично с MutableLiveData и LiveData).

Для отправителей мы создаем MutableSharedFlow:

```
1 | val _eventBus = MutableSharedFlow<Event>()
```

Методом emit можно отправлять данные:

```
1 | _eventBus.emit(event)
```

Обратите внимание, что метод emit - это suspend функция. В обычном коде его не вызывать. Чуть позже мы разберем, зачем это нужно и какие еще есть варианты.

Для получателей мы конвертируем MutableSharedFlow в SharedFlow:

```
1 | val eventBus = _eventBus.asSharedFlow()
```

Получатели используют suspend метод collect, чтобы подписаться на получение данных.

```
1 | eventBus.collect {  
2 |     // ...  
3 | }
```

Учитывайте, что этот вызов никогда не завершится. Потому что SharedFlow все время жив и готов принимать данные от отправителей. Этим он отличается от BroadcastChannel, который мы могли закрыть.

Если со стороны получателя необходимо отменить подписку и перестать получать данные, то мы просто отменяем корутину, в которой вызван collect получателя.

Параметры конструктора

У конструктора MutableSharedFlow есть несколько параметров.

replay

Перед тем, как обсуждать параметр replay, необходимо прояснить один момент.

В работе `MutableSharedFlow` может возникнуть ситуация, когда один из получателей долго обрабатывает полученные данные и не готов пока принимать следующие данные, которые отправитель собирается отправить.

Чтобы понять как это возможно, попробуйте добавить `delay` в вызове `collect` получателя.

```
1 | eventBus.collect {  
2 |     // ...  
3 |     delay(...)  
4 | }
```

В реальных примерах вместо `delay` может быть долго работающая `suspend` функция, например, работа с сетью или диском.

Каждый раз, когда из `Flow` приходят новые данные, для них будет вызываться блок кода, который мы написали в `collect`. И если этот блок не успел выполниться для последних пришедших данных, то получение новых данных будет заблокировано. А это заблокирует отправку новых данных со стороны отправителя. И отправителю придется ждать.

Именно поэтому метод `emit`, который используется для отправки данных в `MutableSharedFlow` является `suspend` функцией. Может возникнуть ситуация, когда ему понадобится приостановить выполнение кода, пока получатели не будут готовы принимать следующую порцию данных.

Чтобы немного смягчить ситуацию, есть возможность включить буфер. Он будет хранить элементы для медленных получателей, чтобы не задерживать всех остальных. Быстрые получатели будут получать данные сразу, минуя буфер.

Возвращаемся к `replay`.

Параметр `replay` включает буфер указанного размера.

```
1 | val _eventBus = MutableSharedFlow<Event>(replay = 3)
```

Кроме буфера, параметр `replay` включает кэш. Этот кэш хранит список последних полученных данных. Размер кэша равен значению `replay`. Соответственно, кэш обновляется с каждым получением новых данных. Каждый новый подписчик сразу получит эти значения.

Важно понимать, что буфер и кэш, хоть и имеют одинаковый размер, но не являются одним и тем же. Буфер может быть пустым, если все получатели достаточно быстрые, но кэш при этом будет полным.

Т.е. буфер нужен, чтобы компенсировать работу медленных получателей. А кэш нужен, чтобы новые получатели сразу смогли получить несколько пришедших ранее данных.

Чтобы посмотреть содержимое кэша, используем `replayCache`:

```
1 | eventBus.replayCache
```

А методом `resetReplayCache` можно очистить кэш:

```
1 | _eventBus.resetReplayCache()
```

Очистка доступна только отправителям, т.е. объекту `MutableSharedFlow`.

extraBufferCapacity

Второй параметр - `extraBufferCapacity`, служит для увеличения размера буфера, заданного параметром `replay`.

Если мы используем и `replay` и `extraBufferCapacity`:

```
1 | val _eventBus = MutableSharedFlow<Event>(replay = 3, extraBufferCapacity = 4)
```

то размер буфера будет равен сумме значений этих параметров: $3+4=7$. Но размер кэша будет равен 3.

`extraBufferCapacity` можно использовать и без `replay`

```
1 | val _eventBus = MutableSharedFlow<Event>(extraBufferCapacity = 4)
```

Размер буфера в этом случае будет равен 4. А размер кэша - 0.

onBufferOverflow

Параметр `onBufferOverflow` задает поведение `SharedFlow`, когда отправитель шлет данные, но буфер уже заполнен.

Возможны три режима.

SUSPEND - метод `emit` на стороне отправителя будет приостанавливать корутину, пока не появятся свободные слоты в буфере. Т.е. пока самый медленный получатель не получит значение, после чего оно будет удалено из буфера.

При этом режиме даже быстрым получателям придется ждать новых данных. Данные просто не смогут пройти минуя буфер. Этот режим используется по умолчанию.

Но для этого режима есть один способ избежать приостановки. В качестве альтернативы `suspend` методу `emit`, есть обычный (не `suspend`) метод `tryEmit`. Он тоже пытается отправить данные в `SharedFlow`. И если `SharedFlow` сразу принял данные (т.е. в буфере есть места), то метод вернет `true`. А если `SharedFlow` сообщает, что буфер заполнен и надо ждать, то данные не будут отправлены, а метод вернет `false`.

DROP_OLDEST - метод `emit` будет удалять из заполненного буфера наиболее старые элементы и добавлять новые. Плюс в том, что методу `emit` больше не придется ждать. Отправка будет мгновенной. С другой стороны, до медленных получателей дойдут не все отправленные данные.

DROP_LATEST - метод `emit` не будет отправлять новые значения, если буфер заполнен. Метод `emit` в этом случае также не приостанавливает корутину и обрабатывает быстро. Но все получатели будут пропускать новые данные, если особо медленные получатели все обрабатывают старые данные.

subscriptionCount

При необходимости мы можем следить за количеством подписчиков SharedFlow. Для этого надо подписаться на StateFlow, который мы получаем в subscriptionCount.

В документации есть хороший пример, как это можно использовать:

```
1 | sharedFlow.subscriptionCount
2 |   .map { count -> count > 0 } // map count into active/inactive flag
3 |   .distinctUntilChanged() // only react to true->false changes
4 |   .onEach { isActive -> // configure an action
5 |       if (isActive) onActive() else onInactive()
6 |   }
7 |   .launchIn(scope) // launch it
```

map преобразует количество подписчиков в boolean флаг (есть подписчики или нет). distinctUntilChanged уберет срабатывания, если значение флага не менялось (чтобы при каждом новом подписчике мы не получали значение true)

В итоге метод onActive будет вызываться при появлении первого подписчика, а onInactive - при отключении последнего.

shareIn

Если у нас есть обычный Flow, и мы хотим сделать из него SharedFlow, то можно использовать shareIn.

В качестве примера можно рассмотреть Flow, который возвращает текущие координаты. Допустим, его нам предоставляет какая-то библиотека.

Под капотом он подключается к системному сервису, получает от него данные в колбэк, и возвращает нам в виде Flow. В какой момент он начнет свою работу? Когда кто-то вызовет collect этого Flow. Но если таких вызовов будет несколько, то для каждого из них будет создан отдельный Flow, который подключится к сервису и зарегистрирует там колбэк. Т.е. сколько подписчиков, столько и подключений к сервису (и колбэков). И все эти Flow будут передавать абсолютно одни и те же данные.

С помощью shareIn можно сделать так, чтобы был только один Flow на всех подписчиков:

```
1 | val locationSharedFlow = locationFlow.shareIn(
2 |     scope = viewModelScope,
3 |     started = SharingStarted.Lazily,
4 |     replay = 3)
```

locationFlow здесь - это обычный Flow, а locationSharedFlow - это SharedFlow. Теперь, независимо от количества подписчиков locationSharedFlow, будет выполнено только одно подключение к системному сервису локации. О том, когда именно оно будет выполнено, читайте чуть дальше.

Рассмотрим параметры метода shareIn.

scope

Чтобы запустить locationFlow и получать от него данные (под капотом shareIn), нужна корутина. А для корутины нужен скоуп. Поэтому от нас требуется его предоставить. Соответственно, пока жив этот scope, будет работать и SharedFlow.

started

С обычным Flow нам понятно, в какой момент он стартует - когда его получатель вызовет collect. Но после конвертации в SharedFlow, может быть несколько получателей. И у нас появляется выбор, когда начинать работу в Flow.

Это определяется параметром started, который может принимать следующие значения:

Eagerly - работа в Flow стартует сразу при создании SharedFlow, даже если еще нет подписчиков. В этом случае данные пойдут в никуда (и в кэш). Flow будет работать, пока не отменим scope.

Lazily - стартует при появлении первого подписчика. Flow будет работать, пока не отменим scope..

WhileSubscribed - стартует при появлении первого подписчика. При уходе последнего подписчика - останавливается. Т.е. отменяется подкапотная корутина, в которой работал оригинальный Flow.

У WhileSubscribed есть свои подпараметры:

stopTimeoutMillis - сколько времени ждать до остановки работы с момента ухода последнего подписчика

replayExpirationMillis - как долго живет replay кэш после остановки работы

replay

Имеет то же значение, что и у SharedFlow. Задает размер буфера и кэша.

Обратите внимание, что нельзя явно задать параметры extraBufferCapacity и onBufferOverflow. По умолчанию они будут иметь значения (64 - replay) и SUSPEND.

Но есть возможность задать эти параметры неявно, с помощью оператора buffer перед shareIn:

```
1 | val locationSharedFlow = locationFlow
2 |     .buffer(capacity = 4, onBufferOverflow = BufferOverflow.DROP_OLDEST)
3 |     .shareIn(...)
```

В получившемся SharedFlow будут применены параметры из buffer.

BroadcastChannel

SharedFlow позиционируется как замена BroadcastChannel. В [официальной документации](#) можно найти инструкцию по миграции:

- конструктор BroadcastChannel(capacity) меняем на MutableSharedFlow(0, extraBufferCapacity=capacity)
- вызовы send и offer меняем соответственно на emit и tryEmit
- код подписчиков меняем на вызов collect

StateFlow

Чем StateFlow отличается от SharedFlow? Про StateFlow можно сказать, что он является частным случаем SharedFlow с определенными параметрами.

В [официальной документации](#) есть даже пример, как создать SharedFlow, который будет иметь поведение, аналогичное StateFlow:

```
1 // MutableStateFlow(initialValue) is a shared flow with the following parameters:
2 val shared = MutableSharedFlow(
3     replay = 1,
4     onBufferOverflow = BufferOverflow.DROP_OLDEST
5 )
6 shared.tryEmit(initialValue) // emit the initial value
7 val state = shared.distinctUntilChanged() // get StateFlow-like behavior
```

Размер буфера и кэша задаем равным 1. DROP_OLDEST указывает на то, что новые данные будут заменять старые. Далее постим начальное значение. А distinctUntilChanged будет отсеивать повторы данных. В результате мы получим SharedFlow с поведением StateFlow.

Т.е. StateFlow это SharedFlow, который удобен для того, чтобы хранить в нем состояние чего-либо. Он всегда хранит в кэше одно (последнее полученное) значение, которое будет получать каждый новый подписчик. А вновь пришедшее значение всегда будет в буфере заменять старое.

Простой пример - прогресс какого-либо действия, например - загрузки файла.

Создаем MutableStateFlow на стороне отправителя:

```
1 val _progress = MutableStateFlow<Int>(0)
```

Мы сразу должны указать начальное значение. Передаем туда 0.

По мере выполнения загрузки, мы постим новые значения с помощью value:

```
1 _progress.value = 10
```

Т.к. интерфейс MutableStateFlow наследует интерфейс MutableSharedFlow, то мы сможем также использовать методы emit и tryEmit для отправки значений. Под капотом они просто вызывают value. Никакой приостановки потока не будет, т.к. используется режим DROP_OLDEST, который просто заменяет значение в буфере и не ждет медленных получателей.

Еще раз упомяну, что важное отличие StateFlow от SharedFlow в том, что если отправитель шлет данные, которое равны предыдущим отправленным данным, то новые данные просто отбрасываются (distinctUntilChanged).

Получателям мы отдаем StateFlow версию:

```
1 val progress = _progress.asStateFlow()
```

Им остается вызвать collect:

```
1 progress.collect {
2     // ...
3 }
```

И здесь они будут получать значения прогресса загрузки.

Чтобы узнать, какое последнее значение отправляли в StateFlow, мы используем value:

```
1 val currentProgressValue = progress.value
```

У SharedFlow был метод replayCache, чтобы посмотреть содержимое кэша. В случае StateFlow этот метод просто вернет список из одного элемента value.

stateIn

Из обычного Flow можно сделать StateFlow с помощью оператора stateIn. Он аналогичен оператору shareIn, который мы рассматривали ранее. На вход мы передаем Scope, started и начальное значение.

```
1 flow {
2     // ...
3 }.stateIn(viewModelScope, Eagerly, 0)
```

ConflatedBroadcastChannel

StateFlow позиционируется как замена ConflatedBroadcastChannel. В [официальной документации](#) можно найти инструкцию по миграции:

- конструктор ConflatedBroadcastChannel(capacity) меняем на MutableStateFlow(initialValue)
- вызовы send и offer меняем на value
- код подписчиков меняем на вызов collect

Пример

В качестве примера, приближенного к реальному, можно рассмотреть код загрузки файлов:

```
1 class DownloadingModel {
2
3     private val _state = MutableStateFlow<DownloadStatus>(DownloadStatus.NOT_REQUESTED)
4     val state = _state.asStateFlow()
5
6     suspend fun download() {
7         _state.value = DownloadStatus.INITIALIZED
8         initializeConnection()
9         processAvailableContent {partialData, downloadedBytes, totalBytes ->
10             storePartialData(partialData)
11             _state.value = DownloadProgress(downloadedBytes.toDouble() / totalBytes)
12         }
13         _state.value = DownloadStatus.SUCCESS
14     }
15 }
```

[Источник](#)

В _state мы храним состояние загрузки. Сначала это INITIALIZED, затем проценты количества загруженных данных и в конце SUCCESS. Можно добавить обработку ошибок и постить ERROR.

Получателям мы предоставляем state, который они будут использовать для отслеживания и, возможно, отображения прогресса на экране.

Заключение

Вся разница между Flow и SharedFlow (StateFlow) сводится к разнице между cold и hot источниками данных.

SharedFlow может иметь несколько получателей (и отправителей) данных, и будет работать в одном экземпляре. А обычный Flow будет создавать новый экземпляр себя под каждого нового подписчика. В RxJava похожую разницу имеют Flowable и Subject.

А также одним из мощных достоинств SharedFlow является то, что он потокобезопасен. Мы свободно можем использовать его в разных корутинах.